HYBRID MPI-CUDA FRAMEWORK FOR HIGH-PERFORMANCE LARGE-SCALE LOG ANALYSIS

Project Report

A performance and correctness study of Serial, MPI-only, CUDA-only, and Hybrid MPI-CUDA approaches for large-scale log analysis.

Technology Stack

C, CUDA 13.4, MPI, Python, Make, Matplotlib and Pandas

Test Environment

Ubuntu 26.04 LTS | NVIDIA RTX 3050 (4 GB VRAM) | 12 CPU cores | 7.6 GB RAM

Abstract

This project presents a Hybrid MPI-CUDA Framework for high-performance analysis of large-scale log files. Four execution variants were implemented: Serial CPU, MPI-only CPU, CUDA-only GPU, and Hybrid MPI-CUDA. Synthetic plain-text application logs containing timestamps, severity levels (ERROR, WARN, INFO and DEBUG), and textual messages were generated at multiple scales. The system counts total log lines and occurrences of each severity level. Correctness testing produced matching counts for the validated workload. Performance measurements showed that MPI-only execution was the fastest approach on the tested hardware and workloads, while CUDA-only and Hybrid MPI-CUDA incurred additional data-transfer, kernel-launch, synchronization and GPU-sharing overhead.

Keywords: MPI, CUDA, GPU acceleration, parallel log analysis, high-performance computing, large-scale data processing

1. Introduction

Large applications continuously generate textual logs for monitoring, debugging, security investigation and system analysis. As log volume grows, sequential scanning can become a processing bottleneck. This project investigates CPU parallelism using MPI, GPU parallelism using CUDA, and a combination of both for large-scale log analysis.

2. Problem Statement

The goal is to efficiently scan large plain-text log files, count total records, identify severity levels, and aggregate ERROR, WARN, INFO and DEBUG counts. The project compares four implementations under the same workload.

3. Objectives

• Generate controlled large-scale synthetic log datasets.
• Implement Serial, MPI-only, CUDA-only and Hybrid MPI-CUDA analyzers.
• Verify correctness through matching aggregate counts.
• Benchmark multiple dataset sizes and MPI process counts.
• Calculate speedup and efficiency and explain the observed performance.

4. Background and Technology

MPI provides process-level parallelism. The file is divided into ranges, each MPI rank analyzes its portion, and MPI_Reduce combines partial results. CUDA provides GPU parallelism; the CUDA kernel scans log data and uses atomic operations for shared counters. The Hybrid version combines MPI decomposition with CUDA processing for each rank's chunk.

5. System Requirements

Component	Specification
OS	Ubuntu 26.04 LTS
CPU	12 cores
GPU	NVIDIA RTX 3050, 4 GB VRAM
RAM	7.6 GB
CUDA	13.4

6. System Architecture

```
Input Log File → MPI Data Partitioning (where applicable) → CPU or CUDA Processing → Local Counts
→ MPI Reduction → Final Results.
```

MPI and Hybrid implementations divide the file among ranks and adjust boundaries around newline characters so log records are not intentionally split. The Hybrid path reads each chunk, transfers it to the GPU, runs the CUDA analyzer, retrieves local counts, and reduces them across ranks.

7. Dataset and Log Format

The project uses synthetic plain-text application logs. A record contains a timestamp, a severity level in square brackets, and a textual message.

```
Example:
2026-09-12T10:15:32 [ERROR] User login successful
2026-09-12T10:15:33 [INFO] User login successful
2026-09-12T10:15:34 [WARN] User login successful
2026-09-12T10:15:35 [DEBUG] User login successful
```

Datasets tested included approximately 100MB, 500MB, 1GB and 2GB-class workloads. The approximately 100MB file was 92,152,406 bytes and contained 1,500,000 lines.

8. Implementation

8.1 Serial CPU: scans the complete file sequentially and counts lines and severity tokens.

8.2 MPI-only: divides the file among MPI ranks, performs CPU analysis independently, and combines counts with MPI_Reduce.

8.3 CUDA-only: copies input data to GPU memory, launches a CUDA kernel, uses GPU threads and atomic counters, and returns aggregate counts.

8.4 Hybrid MPI-CUDA: combines MPI file decomposition with CUDA analysis for each rank.

9. Correctness Verification

For the validated approximately 92MB / 1,500,000-line dataset, the implementations produced matching totals:

Metric	Count
Lines	1,500,000
ERROR	75,056
WARN	224,933
INFO	900,082
DEBUG	299,929

Serial and CUDA matched these values. MPI with 2 and 4 processes and Hybrid with 2 and 4 processes were also checked against the same totals.

10. Experimental Method

Benchmarking was performed using the project benchmark script. MPI process counts of 1, 2, 4 and 8 were tested where available. The 8-process MPI runs required --oversubscribe in this environment. Speedup was calculated as Serial time / Configuration time, and efficiency as Speedup / Number of processes.

11. Performance Results

Measured execution times:

Dataset	Variant	N	Time(s)
100MB	Serial	1	0.1343
100MB	CUDA	1	2.3638
100MB	MPI	1	0.3747
100MB	MPI	2	0.1775
100MB	MPI	4	0.1367
100MB	Hybrid	1	1.1586
100MB	Hybrid	2	1.0894
100MB	Hybrid	4	1.1572
500MB	Serial	1	0.6575
500MB	CUDA	1	1.9245
500MB	MPI	1	0.5478
500MB	MPI	2	0.2672
500MB	MPI	4	0.1725
500MB	MPI	8	0.1277
500MB	Hybrid	1	1.2206
500MB	Hybrid	2	1.0853
500MB	Hybrid	4	1.1537
500MB	Hybrid	8	1.3553
1GB	Serial	1	1.4727
1GB	CUDA	1	2.3270
1GB	MPI	1	1.0356
1GB	MPI	2	0.5215
1GB	MPI	4	0.3440
1GB	MPI	8	0.2424
1GB	Hybrid	1	1.5784
1GB	Hybrid	2	1.4683
1GB	Hybrid	4	1.4095
1GB	Hybrid	8	1.5036
2GB	Serial	1	0.2725
2GB	CUDA	1	2.8062
2GB	MPI	1	0.2193
2GB	MPI	2	0.1176
2GB	MPI	4	0.0797

Dataset	Variant	N	Time(s)
2GB	MPI	8	0.0560
2GB	Hybrid	1	1.0573
2GB	Hybrid	2	1.0832
2GB	Hybrid	4	1.0846
2GB	Hybrid	8	0.6759

12. Results and Discussion

The fastest recorded configuration for 100MB was Serial at 0.1343 s. For the larger recorded datasets, MPI-only with 8 processes gave the lowest measured time: 0.1277 s for 500MB, 0.2424 s for 1GB, and 0.0560 s for 2GB. The corresponding recorded speedups were approximately 5.15×, 6.08× and 4.87× respectively.

MPI performed well because the workload is primarily a straightforward text scan and CPU processes can work on independent file ranges without GPU transfer overhead. CUDA-only was slower because data must be transferred to GPU memory and the kernel requires launch and synchronization overhead. The Hybrid version adds these GPU costs for every MPI rank, while multiple ranks share one RTX 3050 GPU, causing additional contention.

The recorded 2GB Serial time is unusually low compared with the 1GB Serial time. This suggests that filesystem caching, system state or measurement variability may have influenced individual runs. Therefore, the measurements should be presented as recorded benchmark observations rather than a strict linear scaling model.

13. Limitations

• One GPU was available for hybrid testing.
• The GPU has 4GB VRAM.
• CUDA atomic counters can create contention.
• File-system cache and system load can affect timing.
• The benchmark records are individual measurements rather than statistical averages.
• The analysis workload is simpler than complex production log analytics.

14. Conclusion

The project successfully implemented and compared Serial, MPI-only, CUDA-only and Hybrid MPI-CUDA log analysis. Correctness was validated through matching aggregate counts. The recorded performance results show that MPI-only was the best-performing approach on the tested system, particularly for the larger workloads. The project demonstrates that GPU acceleration and hybrid parallelism do not automatically produce the best performance; the workload, data movement and hardware configuration strongly influence the result.

15. Future Work

Future improvements include streaming input in smaller chunks, pinned host memory, asynchronous CUDA streams, overlapping I/O and computation, per-block GPU counters to reduce atomic contention, improved record-oriented CUDA kernels, multi-GPU mapping, repeated benchmark trials with averages and standard deviation, and more complex log analytics such as time-window statistics and anomaly detection.

16. Viva Questions and Answers

What type of logs did you use? Synthetic plain-text application logs with timestamps, ERROR/WARN/INFO/DEBUG levels and messages.

Why synthetic logs? To control size and structure and keep comparisons consistent.

What is MPI? A message-passing model used here to divide the file across CPU processes and reduce their results.

What is CUDA? NVIDIA's GPU computing platform used here for parallel log scanning.

Why was MPI faster than Hybrid? The workload is simple, while Hybrid adds GPU transfers, kernel launches,

synchronization and GPU contention.

How was correctness checked? By comparing line and severity counts across implementations.

What is speedup? Serial execution time divided by configuration execution time.

What is the main conclusion? MPI-only gave the best measured performance on the tested hardware and workloads.

17. Appendix: Project Files and Commands

Main project files include Makefile, README.md, generate_logs.py, benchmark.sh, analyze_results.py, include/cpu_analyze.h, include/kernel.h, and the C/CUDA source files under src/.

```
To reopen the project:
cd ~/hybrid_mpi_cuda_log_analysis

Build:
make

Regenerate analysis:
python3 analyze_results.py

Open graphs:
explorer.exe speedup_report.png
explorer.exe execution_time_report.png
```

18. References

1. NVIDIA CUDA Toolkit documentation and programming guides.
2. MPI Forum, Message Passing Interface standard documentation.
3. Project source code and benchmark scripts.
4. Generated CSV benchmark and analysis outputs.

Report note: Benchmark values are the recorded measurements from the project experiments. Repeated trials are recommended for a formal performance study.